

La disciplina de Ingeniería del Software.

¿Qué comprende?

-

Software

- Es el conjunto de programas, estructuras de datos y la documentación que los acompaña
 - Es conocimiento representado en diferentes niveles de abstracción, teniendo las propiedades de que no se manufactura, no se gasta y se construye a medida.
-

Ingeniería de software

- Según la IEEE es la aplicación de un enfoque sistemático, disciplinado y cuantificable al desarrollo, operación y mantenimiento del software
 - Según el Sommerville es una disciplina de la ingeniería que se ocupa de todos los aspectos de la producción de software, desde la concepción hasta la operación y mantenimiento
 - Según David Parnas es la construcción multi-persona de software multi-versión
-

Qué Comprende la Disciplina

- El **Compromiso con la Calidad** es el fundamento organizacional sobre el cual se apoya todo lo demás
 - El **Proceso** es lo que une las tecnologías, definiendo el marco de trabajo para administrar el proyecto, asegurar la calidad y controlar el cambio
 - Los **Métodos** indican técnicamente cómo se construye el software (comunicación, análisis, diseño, codificación y pruebas)
 - Las **Herramientas** ofrecen soporte automatizado para los Procesos y Métodos.
-

Las Áreas de Trabajo

- Las **Disciplinas Técnicas** incluyen los Requerimientos, el Análisis y Diseño, la Construcción, la Prueba y el Despliegue.
- Las **Disciplinas de Gestión** incluyen la Planificación de Proyecto, el Monitoreo y el Control.
- Las **Disciplinas de Soporte** incluyen la Gestión de la Configuración del Software (SCM), el Aseguramiento de la Calidad (SQA) y Métricas.

Las Actividades del Proceso

- Las **Actividades Estructurales o Genéricas** incluyen la comunicación, la planeación, el modelado, la construcción y el despliegue.
- Las **Actividades Sombrilla** se centran en el control, el seguimiento, la gestión de riesgos, las revisiones técnicas y el aseguramiento de la calidad.

La Crisis del Software

•

Síntomas

- Los **Desvíos de Presupuesto y Tiempo** que hacen que los proyectos cuesten más y tarden el doble de lo planeado
- La **Insatisfacción del Cliente** debido a que el producto final no cumple con lo que el usuario realmente necesita
- La **Mala Calidad y Documentación** que es generada por sistemas propensos a fallar y casi imposibles de entender
- La **Inviabilidad de Mantenimiento** que hace imposible modificar el código para agregar una nueva funcionalidad

La Raíz del Problema

- Los costos principales están en el diseño y la especificación conceptual, no en la producción del archivo

- El Software no se Desgasta como el Hardware, sino que se Deteriora debido a los cambios continuos que introducen efectos colaterales al diseño
- El Software es menos predecible ya que no están sujetos a leyes físicas que limiten o guíen el comportamiento de código
- No existe la producción de Software en masa, prácticamente todo es una pieza única hecha a medida
- Hay algunas fallas que no son de desgaste, sino que exponen errores de diseño o de traducción conceptual

Mitos del Software

- El **Mito de la Gestión** indica que si se agrega más gente al proyecto atrasado va a ayudar a ponerlo al día, pero tiene el efecto contrario, lo retrasa más al darle al equipo más responsabilidades
- El **Mito del Cliente** indica que con una idea general basta, los detalles se ven sobre la marcha, pero los cambios tardíos son mucho más difíciles de cambiar que al comienzo
- El **Mito del Profesional** indica que una vez que el código anda, el trabajo terminó, pero la mayor parte del trabajo se invierte en Mantener y Evolucionar el código.

Disciplinas que Conforman la Ingeniería de Software

- ### Disciplinas Técnicas

- *Ejecutan la construcción directa del producto de software*
- Los **Requerimientos** entienden y especifican qué necesita el cliente
- El **Análisis y Diseño** define la arquitectura, las estructuras de datos y cómo se resuelve técnicamente el problema
- La **Construcción** es la codificación y la traducción del

- diseño a código real
- La **Prueba (Testing)** valida que el software funcione sin errores antes de entregarlo
- El **Despliegue** lleva el software terminado a producción para que el usuario pueda usarlo

Disciplinas de Gestión

- *Se encargan de que el proyecto ocurra dentro de las restricciones de tiempo y presupuesto*
- La **Planificación de Proyecto** consiste en armar el mapa de ruta, estimar costos, tiempos y asignar recursos.
- El **Monitoreo y Control de Proyectos** mide el avance real contra el plan original para corregir desvíos de tiempo.

Disciplinas de Soporte

- *Cruzan de forma transversal el proyecto para darle estabilidad y previsibilidad*
- La **Gestión de Configuración de Software (SCM)** administra las distintas versiones del código y controla los cambios
- El **Aseguramiento de Calidad (SQA)** garantiza que se sigan los procesos y estándares establecidos.
- Las **Métricas** miden el proceso, el proyecto y el producto para tomar decisiones basadas en datos

Ciclos de Vida y su influencia en la Administración de Proyectos de Software

Ciclo de Vida (CdV)

- Es la serie de pasos a través de los cuales progresa un proyecto o producto. Un CdV de proyecto de software es una representación gráfica y particular de un proceso.
- Es un modelo prescriptivo cuyo propósito es establecer el

- orden en que se ejecutan las actividades y definir los criterios de transición para avanzar de una tarea a la siguiente.

Ciclo de Vida del Producto vs Ciclo de Vida del Proyecto

- El **Ciclo de Vida del Producto** abarca la historia del software desde la idea inicial y el plan de negocio, su creación, puesta en operación, actualizaciones y su retiro del mercado.
- El **Ciclo de Vida del Proyecto** ocurre dentro del ciclo de vida del producto y se enfoca en la etapa que va desde la concepción de una versión hasta su entrega inicial.

Influencia en la Administración de Proyectos

- La **Velocidad de Desarrollo** depende de optimizar los pasos y de que cada etapa acerque al objetivo sin dar vueltas
- La **Gestión y Control de Riesgos** permite identificar y mitigar riesgos en momentos tempranos del proyecto.
- El **Seguimiento del Proyecto (Visibilidad)** determina qué tan fácil es para el Project Manager y para el cliente ver avance real
- La **Calidad del Producto Final** define en qué etapas se aplican las pruebas y revisiones de calidad
- Las **Relaciones con el Cliente** se construyen retroalimentando continuamente con el usuario y evitando sorpresas desagradables en la entrega final.

Procesos de Desarrollo Empíricos vs Definidos

-

Procesos Definidos

- Asume que se puede repetir el mismo proceso una y otra

vea, de manera indefinida, y siempre se va a obtener el mismo resultado.

- Se apoya en la predictibilidad del proceso.
- En el contexto del Desarrollo de Software asume que se puede predecir todo al comienzo del proyecto, ignorando que los requerimientos son cambiantes y complejos.

Procesos Empíricos

- Asume que operamos en procesos creativos y complicados con variables altamente cambiantes, por lo tanto los resultados pueden cambiar al repetir el proceso.
- La administración se basa en la inspección frecuente y la adaptación continua, siguiendo el bucle de: Asumir -> Construir -> Retroalimentar -> Revisar -> Adaptar
- En el contexto del Desarrollo de Software requiere experimentar y ajustar el rumbo sobre la marcha ya que el proceso es un esfuerzo intelectual e intangible.

Ciclos de Vida y Procesos de Desarrollo de Software

- **Relación entre un Proceso de Desarrollo y un Ciclo de Vida**

- El **Proceso de Desarrollo** es el conjunto estructurado de actividades, métodos, prácticas, herramientas y personas que se usan para obtener el software.
- El **Ciclo de Vida** es una representación gráfica y particular del proceso de desarrollo, cuya función es ordenar temporalmente las actividades y establecer criterios para pasar a la siguiente fase.

- **Clasificación General de los Ciclos de Vida**

- Los **Secuenciales** ejecutan las fases una única vez y en orden lineal

- Los **Iterativos/Incrementales** dividen el proyecto en ciclos de desarrollo donde se repiten las fases de diseño, construcción y prueba, entregando partes del software de forma gradual.
- Los **Rekursivos** repiten el proceso de forma anidada en diferentes niveles de abstracción del sistema.

Los 6 Modelos de Ciclo de Vida más Utilizados

○

Cascada Puro (Pure Waterfall)

- Los requerimientos se definen completamente al inicio, luego se diseña, se codifica, se prueba y se entrega
- Necesita que los requerimientos sean conocidos y estables, brindando poca flexibilidad ante cambios y sin resultados visibles por parte del cliente hasta el final.

○

Codificar y Corregir (Code-and-Fix)

- No tiene fase de planificación ni de diseño, se programa directamente, se prueba sobre la marcha y se corrige el código una y otra vez.
- Es el modelo más simple que existe, pero genera sistemas poco mantenibles, de baja calidad y con nula visibilidad de avance real.

○

Espiral (Spiral)

- Modelo iterativo orientado al análisis y control de riesgos, donde en cada iteración se evalúan los riesgos del proyecto, se desarrollan prototipos y se planifica la próxima fase.
- Sirve para proyectos grandes, pero tiene un alto costo de gestión.

○

Prototipado Evolutivo (Evolutionary Prototyping)

- Se construye un prototipo con los requerimientos actuales, se lo presenta al cliente para que devuelva feedback, se lo modifica y se repite hasta convertirlo en el producto final
- Es ideal cuando el cliente es indeciso, pero no se

- puede estimar tiempos de entrega.

Entrega por Etapas (Staged Delivery)

- Se definen los requerimientos y el diseño de todo el sistema, pero la construcción y entrega se hace en etapas sucesivas con software funcional y probado
 - Mantiene el control rígido del diseño inicial combinado con la entrega temprana de valor al negocio
-

Entrega Evolutiva (Evolutionary Delivery)

- Se diseña y se construye el núcleo del sistema, se entrega al cliente y las siguientes funcionalidades se van modificando y adaptando a los usuarios.
- Es altamente adaptable a requerimientos cambiantes, pero requiere desarrolladores y cliente sofisticados

Criterios para la Elección de Ciclos de Vida para Proyectos de Desarrollo

•

10 Criterios de Selección

- Si se **Comprenden los Requerimientos** convienen los modelos lineales, pero si son difusos y cambiantes entonces se necesitan los Iterativos
- Si se **Comprende la Arquitectura** se pueden usar modelos lineales, pero si se usan tecnologías nuevas y poco entendidas se usan modelos de espiral
- Si se necesita **Confiabilidad en Producción** los modelos con fases de diseño rígidas al comienzo suelen ser más predecibles que los evolutivos.
- Si se proyecta un **Crecimiento Futuro** los modelos iterativos y evolutivos están pensados para afrontar este tipo de problemas, mientras que los rígidos la suelen sufrir.
- Si el **Riesgo** es muy alto se prefieren modelos iterativos, ya que cada fase se diseña para identificar y mitigar riesgos.

- Si hay **Rigidez de Calendario** entonces los modelos de entrega por etapas son ideales porque permiten recortar funcionalidades de baja prioridad para asegurar la entrega.
 - Si se quiere minimizar los **Gastos de Gestión** entonces el code-and-fix es ideal, pero hay situaciones donde un costo moderado justifica la envergadura del proyecto
 - Si se busca **Adaptabilidad** entonces los modelos evolutivos permiten correcciones profundas a mitad de camino
 - Si se busca **Visibilidad para el Cliente y la Gerencia** entonces los modelos de entrega continua son los mejores, mientras que los secuenciales entregan solo una vez al final.
 - Dependiendo de la **Sofisticación del Equipo** se prefieren los modelos evolutivos para equipos experimentados y esquemas estructurados y entrega por etapas para equipos más novatos.
- .

La Regla de Decisión

- Cuanto más se pueda apegar a un enfoque lineal de forma efectiva, más rápido será el desarrollo
- En caso de incertidumbre de requerimientos y arquitectura, es más rápido y seguro elegir modelos flexibles.

Vínculo Proceso-Proyecto-Producto en la Gestión de Proyecto

.

Definiciones Base

- El **Producto** es el sistema de software a entregar, comprendiendo el código, las estructuras de datos y la documentación que lo acompaña.
- El **Proceso** es el marco de trabajo adaptativo que incluye el conjunto de actividades, tareas, métodos y herramientas que guían a las personas sobre cómo construir software.

- El **Proyecto** es la instancia real y temporal de ejecución del proceso, se estiman los recursos, se gestionan los riesgos y se coordina al equipo para dar vida a un producto.

Vínculo con la Gestión



El Producto dicta el Proceso

- Las características del Producto que se va a construir definen qué Proceso se va a usar
- Los productos críticos de alta confiabilidad demandan procesos rígidos y secuenciales, y los innovadores buscan procesos empíricos y ágiles.



El Proceso estructura al Proyecto

- El Proceso le da al Administrador del Proyecto las herramientas y el esqueleto de trabajo
- Le dice las Actividades Estructurales y las Actividades Sombrilla que se deben ejecutar a lo largo del tiempo para que el proyecto no caiga en un caos



El Proyecto materializa el Producto

- A través de la gestión del Proyecto se ejecutan las fases para modelar el software hasta que se convierte en el Producto terminado listo para producción

.

La Fusión de Producto y Proceso

- Tomar el **Producto** y descomponerlo en sus funciones y características principales
 - Tomar el **Proceso** y utilizar sus actividades de desarrollo
 - Cruzar **Ambas Dimensiones** para estimar el tiempo y el esfuerzo que va a costar aplicar cada paso del Proceso a cada parte del Producto
-

No Silver Bullet Essence and Accidents of Software

•

El Modelo Aristotélico del Software

- Las **Dificultades Accidentales (Representación)** son los problemas técnicos que dificultan plasmar las ideas en la computadora, pero no son propios del software en si.
 - Las **Dificultades Esenciales (Concepto)** es el problema de construir la compleja estructura de ideas entrelazadas, siendo la parte realmente difícil la especificación, el diseño y la validación del constructo conceptual, no el acto de escribir el código.
-

Las 4 Propiedades Irreducibles de la Esencia

- La **Complejidad** implica que al aumentar el tamaño de un sistema se multiplican exponencialmente las interacciones no lineales entre sus componentes.
 - La **Conformidad** implica que debe conformarse y adaptarse a interfaces arbitrarias impuestas por instituciones humanas o sistemas existentes
 - La **Mutabilidad** implican que al ser útil, los usuarios inventan nuevos usos y debe sobrevivir a la evolución del hardware
 - La **Invisibilidad** implica que no tiene representación física, sino que al diagramarlo encontramos múltiples grafos superpuestos, lo que dificulta la capacidad de visualizarlo.
-

Las Falsas Balas de Plata (Ataques a lo Accidental)

- La **Programación Orientada a Objetos** y los **Lenguajes de Alto Nivel** reducen la complejidad sintáctica, pero no modifican la complejidad intrínseca del sistema.
- La **Inteligencia Artificial** facilita la expresión del diseño y determina las buenas prácticas, pero no soluciona el qué construir.
- La **Programación Gráfica** implica hacer diagramas de flujo

y de diseño, pero estos fracasan al intentar representar un software que es inherentemente invisible.

- La **Verificación de Programas** implica probar matemáticamente un software, pero no ataca el problema de depurar los requerimientos iniciales incorrectos.
-

Los Ataques Reales a la Esencia (Hacia dónde ir)

- **Comprar en vez de Construir** es la forma más drástica de no cometer errores de software, ya que adapta los procesos a un paquete existente en vez de construir algo a medida.
- **Refinar los Requerimientos y Prototipado Rápido** para determinar lo que el cliente realmente quiere, validando las interfaces y funcionalidades antes de construir el sistema.
- **Desarrollar Incrementalmente** construyendo un sistema básico funcional y haciéndolo evolucionar orgánicamente hacia donde apunten los requerimientos
- **Cultivar Grandes Diseñadores** para prevenir el uso de malas prácticas, e identificar y promover el diseño de buenas práctica